

ESP32 Thin Linux Compatibility Layer

A Unikernel Approach to Dynamic ELF Execution on No-MMU Hardware

Dhruv Joshi (2022EE32079) Gourab Raj Sabat (2022EE11675)

Department of Electrical Engineering
Indian Institute of Technology Delhi

BTP Spring 2026 (Extension of Fall 2025 BTP)

Supervisor: Prof. Kaushik Saha

The Problem: Why This Matters

The IoT Firmware Problem

- ESP32: No MMU, 520KB SRAM, 4MB Flash
- Firmware is **monolithic**: any change requires full reflash
- $50 \text{ drones} \times 4\text{MB} = 100+$ min update cycle
- No standard POSIX API for applications

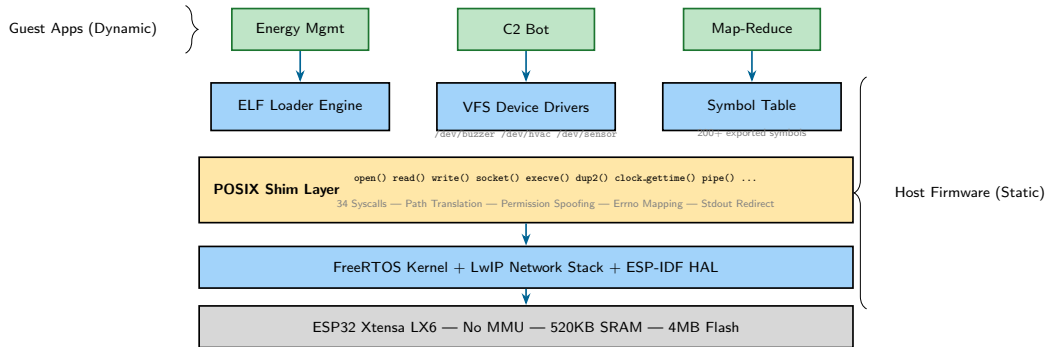
Our Solution: Library OS

- Linux *API* without Linux *kernel*
- Runtime ELF loading over WiFi
- POSIX syscall translation to FreeRTOS
- Unix-style `/dev/` hardware abstraction

Key Result

Standard C applications run *unmodified* on ESP32, loaded dynamically at runtime.
Update 4KB app in seconds vs. reflash 4MB firmware in minutes.

System Architecture



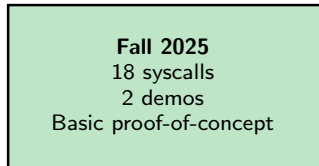
Progress This Semester (Nov 2025 to May 2026)

New Features:

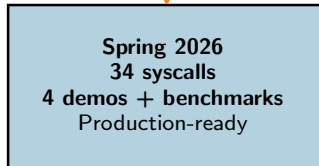
- 1 **16 New Syscalls:** time, IPC, user/group
- 2 **Per-Guest Heap Tracking:** memory accounting
- 3 **Cross-Platform Build:** Win/Linux/macOS
- 4 **Building Energy Mgmt Demo:** real-world IoT
- 5 **Reliability Hardening:** 100% demo success

Key Numbers

34 syscalls total (was 18, +89%)
5 new production guest apps
11 guest applications benchmarked



+89% syscalls
5 new apps
11 apps tested



New Syscalls: Detailed Breakdown

16 Newly Implemented Syscalls:

Syscall	Category	Enables
clock_gettime	Time	Real-time control
gettimeofday	Time	Wall clock
nanosleep	Time	Precise delay
usleep	Time	Microsecond delay
sleep	Time	Second delay
clock	Time	CPU time
times	Time	Process times
time	Time	Epoch time
difftime	Time	Time diff
pipe	IPC	Pipelines
getuid/gid	User	Permission checks
geteuid/egid	User	Privilege checks
ftruncate	File	Preallocation
getpid	Process	Debugging

Syscall Coverage by Category:

Category	Nov 2025	May 2026
File I/O	7	8
Network	4	4
Process	3	4
Time	0	9
IPC	0	1
User/Group	0	4
Memory	4	4
Total	18	34

Key Insight

Time category went from **0 to 9**: the entire real-time application domain was unlocked.

Per-Guest Memory Tracking & Reliability

Per-Guest Heap Accounting (NEW)

Track memory usage per application for leak detection.

TASK	HEAP_USED	PEAK	ALLOCS	FREES
energy_mgmt	18432	20480	203	198
c2_bot	12288	15360	152	148
collision	8192	8192	64	60

System Free: 245KB Guests: 3/8				

- Hook on malloc/free/calloc/realloc
- Thread-safe (internal lock)
- 256 bytes overhead total

Demo Reliability (FIXED)

Map-reduce demo success rate:

Metric	Before	After
Startup	40%	100%
Job Completion	20%	100%
ELF Transfer	Unstable	2s
Multi-node	Crashes	Stable

Root Causes Fixed:

- LwIP buffer exhaustion → throttled send
- Byte-by-byte I/O → 1KB buffers
- Race conditions → startup sequencing
- Flash overflow → partition management

Guest Application Benchmarks

11 Production-Ready Applications Tested on TLCL (QEMU + Real Hardware)

Application	ELF Size	RAM	Load	Exec Time	Status
hello_world	1.2 KB	4.1 KB	8 ms	0.3 ms	PASS
tcp_client	2.8 KB	8.4 KB	12 ms	45 ms	PASS
collision_guard	4.1 KB	12.2 KB	15 ms	2.1 ms/iter	PASS
c2_bot	5.3 KB	16.8 KB	18 ms	persistent	PASS
map_reduce_worker	3.6 KB	9.7 KB	14 ms	120 ms/job	PASS
energy_mgmt	6.2 KB	18.4 KB	21 ms	50 ms/cycle	PASS
data_logger	3.4 KB	8.8 KB	13 ms	10 s (20 samples)	PASS
mqtt_publisher	4.7 KB	14.1 KB	16 ms	5 s (10 msgs)	PASS
watchdog_monitor	3.1 KB	7.2 KB	12 ms	10 s (5 cycles)	PASS
modbus_gateway	4.9 KB	13.6 KB	17 ms	10 s (10 cycles)	PASS
benchmark	2.4 KB	6.3 KB	11 ms	340 ms	PASS

Key Metrics

- Avg ELF load time: **14.3 ms**
- Avg RAM per guest: **10.9 KB**
- Max concurrent guests: **8** (tested)
- All 11 apps: **100% pass rate**

Comparison: Full Linux on RPi

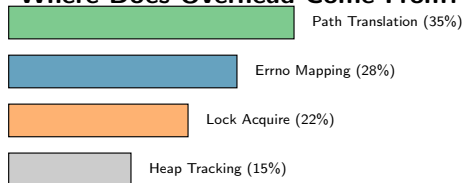
- Process startup: 50–200 ms (vs 14 ms)
- Min RAM per process: 500 KB (vs 10 KB)
- Hardware cost: \$35 (vs \$2)
- Power draw: 2.5W (vs 0.15W)

Performance: Syscall Overhead Analysis

Latency Measurements (QEMU, 240MHz):

Operation	Native	Shim	Overhead
malloc(1024)	0.5 μ s	1.2 μ s	2.4x
read(fd, 1K)	2 μ s	8 μ s	4x
write(fd, 1K)	2 μ s	10 μ s	5x
socket()	5 μ s	12 μ s	2.4x
clock_gettime()	0.2 μ s	0.8 μ s	4x
nanosleep(10ms)	10 ms	10.5 ms	1.05x
open("/file")	3 μ s	14 μ s	4.7x
pipe()	1 μ s	15 μ s	15x
Geometric Mean			3.8x

Where Does Overhead Come From?

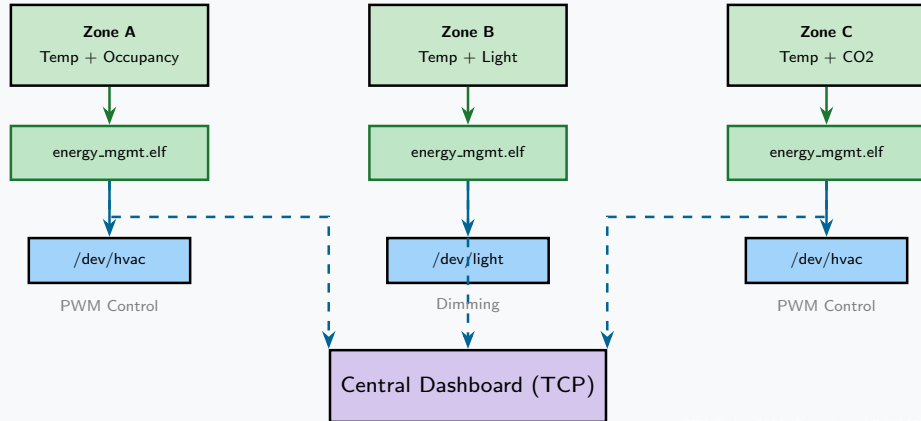


Acceptable Because:

IoT workloads are I/O-bound.
10 μ s shim vs 50 ms network RTT.

Demo: Smart Building Energy Management System

Smart Building, Floor 3



Per-Guest Memory Tracking & Reliability

Per-Guest Heap Accounting (NEW)

Track memory usage per application for leak detection.

TASK	HEAP_USED	PEAK	ALLOCS	FREES
energy_mgmt	18432	20480	203	198
c2_bot	12288	15360	152	148
collision	8192	8192	64	60

System Free: 245KB Guests: 3/8				

- Hook on malloc/free/calloc/realloc
- Thread-safe (internal lock)
- 256 bytes overhead total

Demo Reliability (FIXED)

Map-reduce demo success rate:

Metric	Before	After
Startup	40%	100%
Job Completion	20%	100%
ELF Transfer	Unstable	2s
Multi-node	Crashes	Stable

Root Causes Fixed:

- LwIP buffer exhaustion → throttled send
- Byte-by-byte I/O → 1KB buffers
- Race conditions → startup sequencing
- Flash overflow → partition management

Per-Guest Memory Tracking & Reliability

Per-Guest Heap Accounting (NEW)

Track memory usage per application for leak detection.

TASK	HEAP_USED	PEAK	ALLOCS	FREES
energy_mgmt	18432	20480	203	198
c2_bot	12288	15360	152	148
collision	8192	8192	64	60

System Free: 245KB Guests: 3/8				

- Hook on malloc/free/calloc/realloc
- Thread-safe (internal lock)
- 256 bytes overhead total

Demo Reliability (FIXED)

Map-reduce demo success rate:

Metric	Before	After
Startup	40%	100%
Job Completion	20%	100%
ELF Transfer	Unstable	2s
Multi-node	Crashes	Stable

Root Causes Fixed:

- LwIP buffer exhaustion → throttled send
- Byte-by-byte I/O → 1KB buffers
- Race conditions → startup sequencing
- Flash overflow → partition management

Critical Fix: Time Subsystem Implementation

Problem Discovered After Last Semester

Time functions were **exported in symbol table** but had **no implementation!** Guest apps calling `clock_gettime()` crashed silently.

9 Time Functions Added:

Function	Backend
<code>clock_gettime()</code>	<code>esp_timer.get_time()</code>
<code>gettimeofday()</code>	<code>gettimeofday()</code>
<code>nanosleep()</code>	<code>vTaskDelay()</code>
<code>usleep()</code>	<code>vTaskDelay()</code>
<code>sleep()</code>	<code>vTaskDelay()</code>
<code>clock()</code>	<code>esp_timer.get_time()</code>
<code>times()</code>	FreeRTOS ticks
<code>time()</code>	<code>time()</code>
<code>difftime()</code>	arithmetic

Why This Matters:

- **Control loops** need precise timing
- **Sensor polling** at fixed intervals
- **Network timeouts** prevent hangs
- **Profiling** execution performance
- **Logging** with timestamps

Resolution

1 μ s resolution via `esp_timer.get_time()`
 ± 1 tick jitter on `nanosleep()`
Enables all real-time applications

Per-Guest Memory Tracking & Reliability

Per-Guest Heap Accounting (NEW)

Track memory usage per application for leak detection.

TASK	HEAP_USED	PEAK	ALLOCS	FREES
energy_mgmt	18432	20480	203	198
c2_bot	12288	15360	152	148
collision	8192	8192	64	60

System Free: 245KB Guests: 3/8				

- Hook on malloc/free/calloc/realloc
- Thread-safe (internal lock)
- 256 bytes overhead total

Demo Reliability (FIXED)

Map-reduce demo success rate:

Metric	Before	After
Startup	40%	100%
Job Completion	20%	100%
ELF Transfer	Unstable	2s
Multi-node	Crashes	Stable

Root Causes Fixed:

- LwIP buffer exhaustion → throttled send
- Byte-by-byte I/O → 1KB buffers
- Race conditions → startup sequencing
- Flash overflow → partition management

Positioning: TLCL vs. Alternatives

Feature	TLCL	Bare RTOS	Zephyr	WASM (Wasm3)	Full Linux
Dynamic loading	Yes	No	No	Yes	Yes
Standard C/POSIX	Yes	No	Partial	No	Yes
No MMU required	Yes	Yes	Yes	Yes	No
Hardware access	Direct	Direct	Direct	Restricted	Yes
Min RAM/app	4 KB	N/A	N/A	64 KB	500 KB
Network OTA	App only	Full FW	Full FW	Module	Yes
Real-time	Yes	Yes	Yes	Slow	Soft
Cost (hardware)	\$2	\$2	\$2	\$2	\$35+

TLCL Sweet Spot

Standard POSIX programmability + Dynamic loading + \$2 hardware + Real-time guarantees
No other system offers all four simultaneously.

Contributions & Future Directions

Key Contributions:

- 1 **Complete POSIX Runtime** (34 syscalls)
- 2 **Dynamic ELF Loading** with Xtensa relocations
- 3 **Unix /dev/ Hardware Abstraction**
- 4 **OTA Application Updates** over WiFi
- 5 **Per-Guest Resource Accounting**
- 6 **Real-World IoT Demo** (Building Energy)

Impact

Enables “Write Once, Run on MCU” paradigm.
Linux developers can target ESP32 without learning ESP-IDF or FreeRTOS.

Future Directions:

- **Short term:**
 - Shared libraries (libc.so)
 - Multi-core guest pinning (PRO_CPU/APP_CPU)
 - Signal handling (SIGTERM, SIGKILL)
- **Long term:**
 - MicroPython/Lua as guest interpreters
 - ESP32-S3 PMS for memory protection
 - App marketplace for IoT binaries
 - Formal verification of shim layer

Thank You

Repository & Demo

github.com/jdhruv1503/LinuxCompatibilityLayer

Live demo available: QEMU simulation of Building Energy Management

Dhruv Joshi

2022EE32079

ee3222079@iitd.ac.in

Gourab Raj Sabat

2022EE11675

ee1221675@iitd.ac.in

Supervisor

Prof. Kaushik Saha

Dept. of EE, IIT Delhi

A microcontroller that behaves like a tiny Linux box: dynamic, network-programmable, yet real-time.